

1536. Minimum Swaps to Arrange a Binary Grid

Solved 

Medium

 Topics

 Companies

 Hint

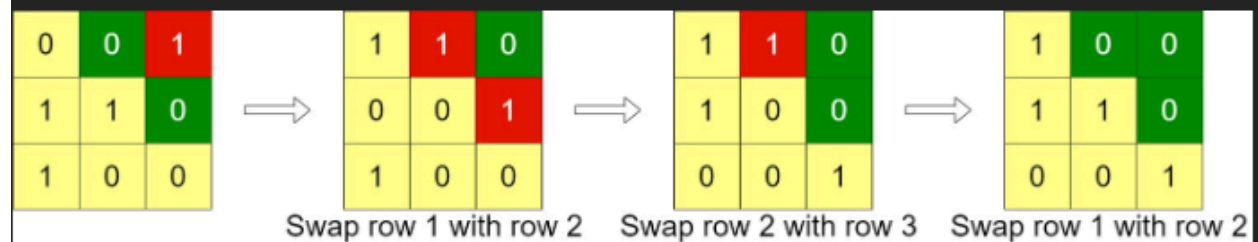
Given an $n \times n$ binary `grid`, in one step you can choose two **adjacent rows** of the grid and swap them.

A grid is said to be **valid** if all the cells above the main diagonal are **zeros**.

Return *the minimum number of steps* needed to make the grid valid, or **-1** if the grid cannot be valid.

The main diagonal of a grid is the diagonal that starts at cell $(1, 1)$ and ends at cell (n, n) .

Example 1:



Input: grid = [[0,0,1],[1,1,0],[1,0,0]]

Output: 3

Example 2:

0	1	1	0
0	1	1	0
0	1	1	0
0	1	1	0

Input: grid = [[0,1,1,0],[0,1,1,0],[0,1,1,0],[0,1,1,0]]

Output: -1

Explanation: All rows are similar, swaps have no effect on the grid.

Example 3:

1	0	0
1	1	0
1	1	1

Input: `grid = [[1,0,0],[1,1,0],[1,1,1]]`

Output: `0`

Constraints:

- `n == grid.length == grid[i].length`
- `1 <= n <= 200`
- `grid[i][j]` is either `0` or `1`

Python:

class Solution:

def minSwaps(self, grid: List[List[int]]) -> int:

n = len(grid)

zeros = list(map(lambda r: (r[::-1] + [1]).index(1), grid))

swaps = 0

for i in range(n):

j = (zeros + [n]).index(next(filter(lambda v: v >= n - 1 - i, zeros + [n])))

if j == len(zeros): return -1

swaps += j

zeros.pop(j)

```
return swaps
```

JavaScript:

```
const minSwaps = grid => {  
  const n = grid.length;  
  let Z = grid.map(r => n - 1 - r.lastIndexOf(1));  
  let swap = 0;  
  
  for (let i = 0; i < n; i++) {  
    const req = n - 1 - i;  
    const j = Z.findIndex(c => c >= req);  
  
    if (j === -1) return -1;  
  
    swap += j;  
    Z.splice(j, 1)[0];  
  }  
  
  return swap;  
};
```

Java:

```
class Solution {  
  public int minSwaps(int[][] grid) {  
    int n = grid.length;  
    List<Integer> zeros = new ArrayList<>();  
    for (int[] row : grid) {  
      int trail = 0;  
      for (int j = n - 1; j >= 0 && row[j] == 0; j--)  
        trail++;  
      zeros.add(trail);  
    }  
  
    int swaps = 0;  
    for (int i = 0; i < n; i++) {  
      int j;  
      for (j = 0; j < zeros.size(); j++)  
        if (zeros.get(j) >= n - 1 - i) break;  
  
      if (j == zeros.size()) return -1;  
      swaps += j;  
      zeros.remove(j);  
    }  
  
    return swaps;}}}
```